INTERVIEW PREP SERIES

CSS

Interview Questions & Easy Answers

---

53 questions · 19 marked ★ most asked

Covers selectors, specificity, the box model, positioning, Flexbox, Grid, responsive design, animations, units and common "how would you center a div" style questions.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is CSS and what are the ways to add it? ★ Most asked

CSS (Cascading Style Sheets) controls how HTML looks: colours, layout, fonts and spacing. Three ways:

- **Inline**: `<p style="color: red">` (highest priority, hard to maintain).
- **Internal**: a `<style>` tag in the `<head>`.
- **External**: a separate `.css` file linked with `<link rel="stylesheet" href="style.css">` (best practice).

Q2. What does "cascading" mean in CSS?

When more than one rule applies to the same element, CSS decides which one wins based on importance (`!important`), specificity, and source order (later rules win if all else is equal). This flow of rules is the cascade.

Q3. What is specificity? ★ Most asked

Specificity is a score that decides which CSS rule wins when two rules target the same element. From highest to lowest:

- Inline styles (1,0,0,0)
- IDs (0,1,0,0)
- Classes, attributes, pseudo-classes (0,0,1,0)
- Elements and pseudo-elements (0,0,0,1)

Example: `#nav .link` beats `.nav .link`. `!important` overrides all of these, but avoid it.

Q4. What are the types of selectors?

- Universal `*`, element `p`, class `.box`, id `#main`
- Group `h1, h2`
- Descendant `div p`, child `div > p`, adjacent sibling `h1 + p`, general sibling `h1 ~ p`
- Attribute `input[type="text"]`
- Pseudo-class `a:hover`, `li:nth-child(2)`
- Pseudo-element `p::first-line`, `::before`

Q5. What is the difference between pseudo-classes and pseudo-elements? ★ Most asked

A pseudo-class selects an element in a special **state**: `:hover`, `:focus`, `:first-child`, `:nth-child(odd)`. A pseudo-element styles a **part** of an element or adds content: `::before`, `::after`, `::first-letter`, `::placeholder`. Pseudo-elements use double colons.

Q6. What is inheritance in CSS?

Some properties pass from a parent to its children automatically, like `color`, `font-family` and `line-height`. Others, like `margin`, `padding`, `border` and `width`, are not inherited. You can force it with `inherit`.

Box model

Q7. Explain the CSS box model. ★ Most asked

Every element is a rectangular box made of four layers from inside to outside:

- **Content**: the text or image.

- **Padding**: space between content and border.
- **Border**: the line around the padding.
- **Margin**: space outside the border, separating it from other elements.

Interview tip: Draw the four boxes on paper if they ask. It shows clear understanding.

Q8. What is `box-sizing: border-box`? ★ Most asked

By default (`content-box`), `width` only sets the content width, and padding and border are added on top. With `border-box`, the width includes padding and border, so a 200px box stays 200px. Most projects set it globally:

```
*, *::before, *::after { box-sizing: border-box; }
```

Q9. What is the difference between margin and padding?

Padding is space inside the border (it gets the element's background colour). Margin is space outside the border (always transparent). Margins can be negative; padding cannot.

Q10. What is margin collapsing?

When two vertical margins meet (like a bottom margin of one paragraph and the top margin of the next), they combine into one margin equal to the bigger one instead of adding up. It only happens vertically, not horizontally, and not in flex or grid containers.

Display and positioning

Q11. What is the difference between `display: none`, `visibility: hidden` and `opacity: 0`? ★ Most asked

- `display: none` removes the element completely; it takes no space.
- `visibility: hidden` hides it but it still takes space.
- `opacity: 0` makes it transparent; it takes space and can still be clicked.

Q12. What is the difference between `inline`, `block` and `inline-block`? ★ Most asked

- `block`: new line, full width, width/height work.
- `inline`: stays in the line, width/height do not work, vertical margins are ignored.
- `inline-block`: stays in the line like inline, but width, height and all margins work.

Q13. Explain the `position` property values. ★ Most asked

- `static`: default, normal flow; top/left do not work.
- `relative`: moves relative to its normal position; the original space is kept.
- `absolute`: removed from the flow and positioned relative to the nearest positioned (non-static) ancestor.
- `fixed`: positioned relative to the browser window; stays when scrolling (sticky navbars, chat buttons).
- `sticky`: acts like relative until you scroll to a point, then sticks like fixed.

Q14. What is `z-index`?

`z-index` controls which element appears on top when elements overlap. Higher numbers are in front. It only works on positioned elements (not static) or flex/grid items.

Q15. What is the difference between `absolute` and `fixed` positioning?

Absolute is positioned relative to the nearest positioned parent and scrolls with the page. Fixed is positioned relative to the viewport and stays in the same place when you scroll.

Q16. What are floats and how do you clear them?

`float: left/right` pushes an element to one side so text wraps around it. Parents may collapse because floats leave the normal flow. Fix with `clear: both` on a following element or `display: flow-root` on the parent. Today we use Flexbox/Grid for layout instead.

Q17. What is the `overflow` property?

It controls what happens when content is bigger than its box: `visible` (default, spills out), `hidden` (cut off), `scroll` (always shows scrollbars), `auto` (scrollbars only when needed).

Flexbox

Q18. What is Flexbox? ★ Most asked

Flexbox is a one-dimensional layout system for arranging items in a row or a column. You set `display: flex` on the parent, and its children become flex items that can grow, shrink and align easily.

Q19. What are the main Flexbox properties? ★ Most asked

On the container:

- `flex-direction`: row | column
- `justify-content`: aligns items on the main axis (flex-start, center, space-between, space-around, space-evenly)
- `align-items`: aligns items on the cross axis (stretch, center, flex-start, flex-end)
- `flex-wrap`: wrap items to the next line
- `gap`: space between items

On the items: `flex-grow`, `flex-shrink`, `flex-basis` (shorthand `flex: 1`), `align-self`, `order`.

Q20. What is the difference between `justify-content` and `align-items`?

`justify-content` aligns items along the main axis (horizontal for a row). `align-items` aligns them along the cross axis (vertical for a row). If you change `flex-direction` to column, the axes swap.

Q21. What does `flex: 1` mean?

It is shorthand for `flex-grow: 1; flex-shrink: 1; flex-basis: 0`. All items with `flex: 1` share the available space equally.

Q22. How do you center a div horizontally and vertically? ★ Most asked

The easiest ways:

```
/* Flexbox */
.parent { display: flex; justify-content: center; align-items: center; height:
100vh; }

/* Grid */
.parent { display: grid; place-items: center; height: 100vh; }

/* Absolute + transform */
```

```
.child { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -
50%); }
```

For horizontal-only centering of a block with a width: `margin: 0 auto;`.

Grid

Q23. What is CSS Grid? ★ Most asked

Grid is a two-dimensional layout system that handles rows and columns at the same time. It is great for full page layouts and card galleries.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

Q24. What is the difference between Flexbox and Grid? ★ Most asked

Flexbox is one-dimensional (a row OR a column) and is content-first, ideal for navbars, buttons and aligning items. Grid is two-dimensional (rows AND columns) and is layout-first, ideal for page layouts and galleries. They are often used together.

Q25. What is the `fr` unit?

`fr` means "fraction of the free space" in a grid. `grid-template-columns: 1fr 2fr` makes the second column twice as wide as the first.

Q26. How do you make a responsive grid without media queries?

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

The browser fits as many 200px-minimum columns as possible and stretches them to fill the row.

Responsive design and units

Q27. What is responsive design? ★ Most asked

Designing pages that adapt to every screen size: mobile, tablet and desktop. Tools: the viewport meta tag, fluid widths (%), flexible units (rem, vw), Flexbox/Grid, responsive images and media queries.

Q28. What are media queries? ★ Most asked

Media queries apply CSS only when a condition is true, usually screen width.

```
.cards { display: grid; grid-template-columns: 1fr 1fr 1fr; }
@media (max-width: 768px) {
  .cards { grid-template-columns: 1fr; }
}
```

Q29. What is the mobile-first approach?

You write base styles for small screens first, then add `min-width` media queries for bigger screens. Mobile CSS stays simple and fast, and it matches how most users browse today.

Q30. What is the difference between `px`, `em`, `rem`, `%`, `vh` and `vw`? ★ Most asked

- `px`: fixed size.

- `em`: relative to the parent's font size (can compound when nested).
- `rem`: relative to the root (`html`) font size; predictable, best for fonts and spacing.
- `%`: relative to the parent's size.
- `vh`/`vw`: 1% of the viewport height/width.

Q31. What is the difference between `min-width`, `max-width` and `width`?

`width` sets a fixed size. `max-width` says "never bigger than this" and lets the element shrink on small screens, which is great for responsiveness (`max-width: 100%` on images). `min-width` says "never smaller than this".

Visual effects and animations

Q32. What is the difference between transition and animation? ★ Most asked

A **transition** smoothly changes a property from one value to another when the state changes (like on hover); it needs a trigger. An **animation** uses `@keyframes`, can have many steps, can loop and can start on its own.

```
.btn { transition: background 0.3s ease; }
.btn:hover { background: navy; }

@keyframes spin { from { transform: rotate(0); } to { transform: rotate(360deg); } }
.loader { animation: spin 1s linear infinite; }
```

Q33. What is the `transform` property?

It changes an element's shape or position without affecting the layout around it: `translate()`, `rotate()`, `scale()`, `skew()`. Transforms and opacity are the cheapest properties to animate because they don't trigger layout.

Q34. What are CSS variables (custom properties)?

Reusable values defined with `--name` and used with `var()`.

```
:root { --primary: #1f4e79; }
.btn { background: var(--primary); }
```

They can be changed at runtime with JS and are commonly used for themes like dark mode.

Q35. What are `::before` and `::after` used for?

They insert decorative content before or after an element's content using the `content` property, e.g. icons, quotes, overlays or custom bullets, without adding extra HTML.

Q36. How do you add a gradient background?

```
background: linear-gradient(to right, #ff7e5f, #feb47b);
background: radial-gradient(circle, white, skyblue);
```

Q37. What is the difference between `rgba` and `opacity`?

`rgba(0,0,0,0.5)` makes only that colour semi-transparent (like the background), while `opacity: 0.5` makes the entire element, including its text and children, semi-transparent.

Advanced and practical

Q38. What are CSS preprocessors?

Tools like **Sass/SCSS** and **Less** that add variables, nesting, mixins and functions to CSS. They compile into normal CSS. Native CSS now supports variables and nesting too.

Q39. What is Tailwind CSS and how is it different from Bootstrap? ★ Most asked

Tailwind is a utility-first framework: you style by combining small classes like `flex p-4 text-center bg-blue-500`. Bootstrap gives ready-made components like `.btn` and `.navbar`. Tailwind gives more design freedom; Bootstrap is faster for standard-looking UIs.

Q40. What is BEM?

BEM (Block, Element, Modifier) is a class naming convention that keeps CSS organised: `.card`, `.card__title`, `.card--featured`. It avoids naming clashes in large projects.

Q41. What is the difference between `reset.css` and `normalize.css`?

Reset removes all browser default styles (margins, heading sizes) so you start from zero. Normalize keeps useful defaults and only fixes inconsistencies between browsers.

Q42. What are reflow and repaint?

Reflow (layout) happens when the size or position of elements changes and the browser recalculates the layout. It is expensive. **Repaint** happens when only looks change (like colour) and is cheaper. Animate `transform` and `opacity` to avoid reflow.

Q43. How do you improve CSS performance?

Minify CSS, remove unused CSS, avoid very deep selectors, load critical CSS first, avoid `@import` chains, and animate only `transform`/`opacity`.

Q44. What is `object-fit`?

It controls how an `<img>` or `<video>` fills its box. `cover` fills the box and crops the extra, `contain` shows the whole image with empty space if needed. Useful for profile pictures and cards.

Q45. What are `nth-child` and `nth-of-type`?

`li:nth-child(2)` selects the second child if it is an `li`. `p:nth-of-type(2)` selects the second `p` among its siblings, ignoring other tags. `nth-child(odd)` or `(2n)` is used for striped tables.

Q46. How do you make text truncate with "..."?

```
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
```

Q47. What is the `:root` selector?

`:root` targets the top-level `<html>` element but with higher specificity. It is the usual place to define global CSS variables.

Q48. How would you build a dark mode?

Define colours as CSS variables in `:root`, then override them for a dark theme, either with a `.dark` class on `<html>` or the `prefers-color-scheme: dark` media query. Toggle the class with JavaScript and save the choice in localStorage.

More frequently asked

Q49. What is the difference between `visibility: hidden` and `display: none` for accessibility?

Both hide the element from screen readers. If you want text hidden visually but still read by screen readers (like a "skip to content" label), use a visually-hidden class (position absolute, 1px size, clip) instead.

Q50. What is the difference between `position: relative` on a parent and `absolute` on a child? ★ Most asked

This is the most common positioning pattern. The parent gets `position: relative` (without moving it) so it becomes the reference box. The child with `position: absolute; top: 0; right: 0;` is then placed at the parent's top-right corner, e.g. a badge on a card or a close button on a modal.

Q51. What is `calc()`?

A CSS function that does math with mixed units: `width: calc(100% - 250px);` makes a main area that fills the space next to a 250px sidebar.

Q52. What are `clamp()`, `min()` and `max()`?

`clamp(min, preferred, max)` keeps a value within a range, great for fluid font sizes: `font-size: clamp(1rem, 2.5vw, 2rem);`. `min()` picks the smaller value and `max()` the larger.

Q53. What is the `:has()` selector?

A "parent selector" that styles an element based on what it contains: `.card:has(img)` styles only cards that contain an image. It's supported in all modern browsers.